EIE330 FPGA-based System Design — Lab 1 Requirements

Lab 1 contains two tasks:

- **Lab 1-1 Full Adder**
- **Lab 1-2 One-bit Digital Comparator**

Do **both** tasks. For **each** task, follow the development flow taught in the lecture from start to finish — preliminary research, FPGA solution, coding, functional simulation, time analysis, and testing on the development board — and write **one report per task**.

---

1. Task 1-1: Full Adder

Use the development board to implement a **full adder**:

- **3 buttons** as 3 individual inputs:
  - one button is the addend `A`,
  - one button is the augend `B`,
  - one button is the carry-in `Cin` from the lower bit;
- **2 LEDs** as 2 outputs:
  - one LED presents the sum `S`,
  - one LED presents the carry-out `Cout`.

Required behavior, as a user of the board sees it:

- **Pressing** a button means that input is logic `1`; leaving it released means logic `0`.
- An LED **on** means the corresponding output bit is `1`; LED off means `0`.
- All **8** input combinations must be verified against the full-adder truth table.

2. Task 1-2: One-bit Digital Comparator

Use the development board to implement a **one-bit digital comparator** with 2 buttons as inputs (`A`, `B`) and 3 LEDs as outputs:

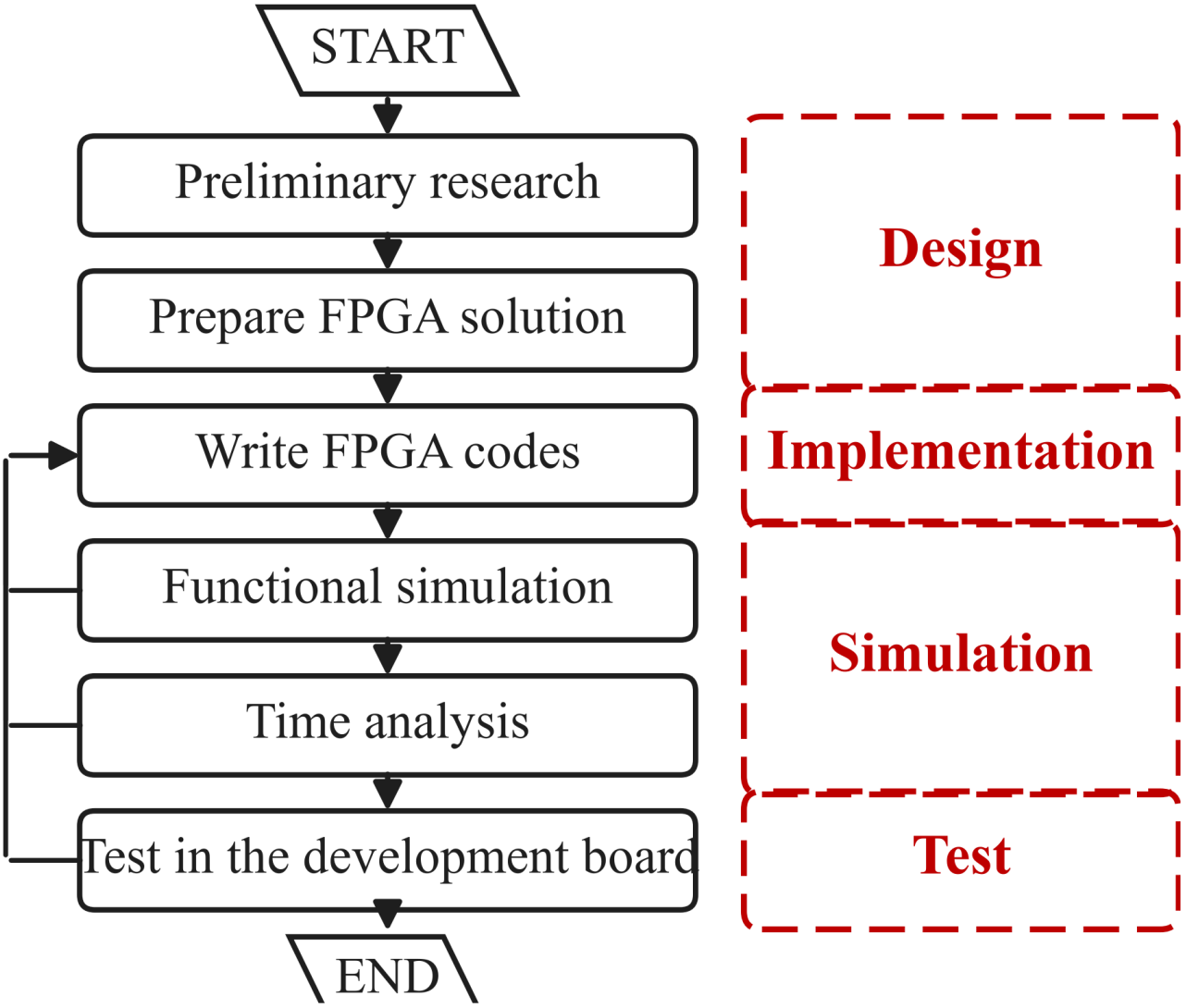
- when `A < B`, only the **1st** LED is on, the others are off;
- when `A = B`, only the **2nd** LED is on, the others are off;
- when `A > B`, only the **3rd** LED is on, the others are off.

Same conventions as Task 1-1: pressed = logic `1`, LED on = output `1`. All **4** input combinations must be verified.

---

3. Required work — follow the development flow

Do every step below **for each task**, in order. The flow is the one from the lecture:



Whenever a verification step fails, **go back** — fix the code, or revisit the design — and come down the flow again. Skipping ahead with a known problem costs you more time than looping back.

Step 1: Preliminary research

Understand what you are asked to build before designing anything.

- Identify the experiment **objectives** and write down **every** functional requirement: the inputs, the outputs, and the required behavior for **every input combination**.
- Identify the **required devices** (FPGA chip, buttons, LEDs) and analyze the **design constraints**, quantified where possible:
  - the electrical behavior of the buttons and LEDs (note: they are active-low on the development board);
  - how many pins and logic elements the FPGA chip provides — look it up in the **datasheet** and cite it.
- Do a short **literature review**: the FPGA datasheet, the tool documentation, and background reading on full adders / comparators. Record what you actually used — you will cite it in the report.

Output of this step: a requirement list in which **every item can be checked later** by a specific test.

Step 2: Prepare FPGA solution

Turn the requirements into a concrete, implementable design — on paper, before any code.

- Translate the requirements into a **design specification**: module name, port list (signal names, input/output, active level of each port), the truth table, and the **expected input/output waveforms** drawn before coding.
- Draw the **structure block diagram** with every signal labeled. Decide how many modules you need and why (a top-down design may still be a single module for these small circuits — if so, justify it).
- Write down your **design decisions with reasons**, for example:
  - combinational or sequential logic — why?
  - how do you handle the active-low buttons and LEDs so that the visible behavior meets the requirements in Sections 1 and 2 — and why that way?
  - which description style do you use (structural / dataflow / behavioral), and why is it appropriate here?
- Check **traceability**: every requirement from Step 1 must map to something in this design, and everything in the design must come from a requirement.

Step 3: Write FPGA codes

- Organize the project first: design codes in an `RTL/` folder, and testbench codes in a `sim/` folder.
- Write the Verilog following the coding rules in lectures: consistent naming, clean indentation, meaningful comments, synthesizable constructs only, no latches, no multiple drivers.
- **Verify the syntax** with iverilog: the code must compile cleanly.
- **Verify the implementation with yosys**: generate the RTL view and compare it against your block diagram — same ports, same logic, nothing extra; and read the resource numbers (wires, cells, memories) to see what was inferred. Warnings from iverilog or yosys usually mean real problems: fix them, do not ignore them.

Step 4: Functional simulation

Verify the **functions** of your design in simulation, before touching the board.

- Decide your **verification objectives first**: which cases must be checked for this design to be considered correct?
- Write the testbench: `reg` signals drive the inputs, `wire` signals observe the outputs, an `initial` block applies every input combination step by step. Keep the expected output of each step written next to its stimulus.
- The testbench must contain `$dumpfile` / `$dumpvars` so that a `.vcd` file is produced — GTKWave and the report figures both read it.
- Use GTKWave to **debug**: find where the actual behavior first deviates from your expectation.

Step 5: Time analysis

- Compare the simulated waveforms against the **expected waveforms you drew in Step 2**, row by row. They must be identical for every truth-table case; if not, locate the cause (design? code? testbench?) and loop back.
- State clearly what this analysis does **not** cover: the functional simulation is **zero-delay**. Real propagation delay, and whether the design still works under it, can only be confirmed on the board in Step 6.

- Produce the **waveform figures for your report** from the simulation data — **GTKWave screenshots are not acceptable in a report**. You may convert the VCD file to CSV with `vcd2csv.py` (get it here; it needs the `vcdvcd` Python package).

Step 6: Test in the development board

- **Plan the pins**: map each module port to a board signal (`B2`–`B5`, `led1`–`led5`, …); everything else stays unconnected. Write the mapping down — it goes into the report.
- **Verify every truth-table row on the board, one by one**: press the buttons, observe the LEDs, check the behavior against the specification. Record a **video of the board window** as your verification record; screenshots are optional supporting evidence.
- If the board behavior differs from simulation, debug and loop back to Step 3.

Know the limits of your tools

Each tool in this flow proves something different, and none proves everything: functional simulation is zero-delay and idealized; yosys performs *generic* synthesis, not a real FPGA implementation; the simulator models the real board but is not the real board — final confirmation always requires real hardware. In your report, show that you understand what each tool can and cannot guarantee.

---

4. Report requirements — one report per task

Write **one report for Task 1-1 and one for Task 1-2**. The report tells the story of your development flow; structure it as follows.

a) Introduction

- Background: what the circuit is (for 1-1: what a full adder is, and how it differs from a half adder; for 1-2: what a comparator does), and where it is used.
- The purpose of this experiment.
- A brief overview of your design and your key results.
- What each group member contributed: use a list to show each member's contributions, with names and student IDs.

b) Requirements analysis

- The objectives and the **complete** functional requirements, including the full truth table.
- The design constraints, quantified: the electrical behavior of the buttons/LEDs, whether a clock is needed, the FPGA resources from the datasheet.
- The design specification you derived: ports with names, directions, and active levels; the expected waveforms.
- Make every requirement traceable — a reader should be able to follow each requirement to a spec item, a design element, and later a test case.

c) System design

- The structure block diagram, with every signal labeled.
- The expected input/output waveforms.

- Your design decisions and their justification (combinational vs sequential, active-low handling, description style).

d) Implementation

- Your final Verilog code, as readable text (monospace font), following the coding rules. **You cannot put all your code directly into this part.** Only show the idea of how you translated the design into code. **All code should be put on GitHub, with the link given in the report.**
- Evidence that the code passes the syntax check and yosys synthesis **without warnings**.
- The yosys RTL view, with your interpretation: does it match your block diagram? What do the `stat` numbers say? Any latches or unexpected logic?

e) Testing and results

- Your verification plan: which cases you test, and why those cases suffice.
- How your testbench works: the stimuli, the expected outputs, and how you compare expected vs actual. **You cannot put the testbench code directly into this part either; it goes on GitHub with the rest of the code.**
- The results: expected vs actual outputs for every case, including important internal variables where relevant; explain any difference you observed.
- Waveform figures plotted from the simulation data (see Step 5), each numbered, captioned, and referenced in the text.
- The on-board verification: an itemized checklist of every case you tested on the development board. A **video recording** of the board window is **required** — upload it to a video website and give the link in the report; screenshots inserted in the report are optional supporting evidence.

f) Discussion *(optional but encouraged)*

- An honest discussion of the **limitations of the tools** you relied on.
- Bugs or problems you met: how you found them, what the waveform or RTL view showed, how you fixed them.
- Problems you could not solve in the given time, and your possible solutions.
- Interesting results or observations.

g) Conclusion

- Summarize the design and all verification results.
- Summarize the key discussion points, if any.

h) References

- Everything you cite, in **IEEE format** (IEEE Reference Guide) — the datasheet, tool documentation, and any tutorial you actually used.

Figure, table, and code standards

- Every figure and table is numbered, captioned, and referred to in the text.
- Waveforms are **plotted from data**. GTKWave screenshots are debugging aids and are not accepted as report figures.
- Board-verification records are the opposite: a **video link is required**, and screenshots of the development-board window may be inserted as optional supporting evidence.

- Short code (a line or two) may appear as text, but not as screenshots. Longer code must be given as a GitHub link.

---

5. What to submit

The reports of **all tasks** must be combined into **one** PDF file. The report of **each** task must have **an individual cover page** with the task name, your names, and student IDs.

Plagiarism — of code or of report text — scores zero and is handled under the academic integrity rules.

If you use AI, you must comply with the **MUST AI Policy and Literacy** and the **Guidelines on Artificial Intelligence Application and Ethical Governance**, and state in your report which parts of the work were done with AI.